Collaborative Discussion 1: Structural Design Patterns

Adapter Pattern

In the first scenario, I implemented the Adapter Pattern for cyber security log monitoring. I created a LogAdapter to connect a legacy log collector with a modern monitoring system.

The adapter converts the legacy string-based logs into the standard format used by the get_events() interface. This allows both legacy and modern events to be processed in the same way without modifying the legacy component.

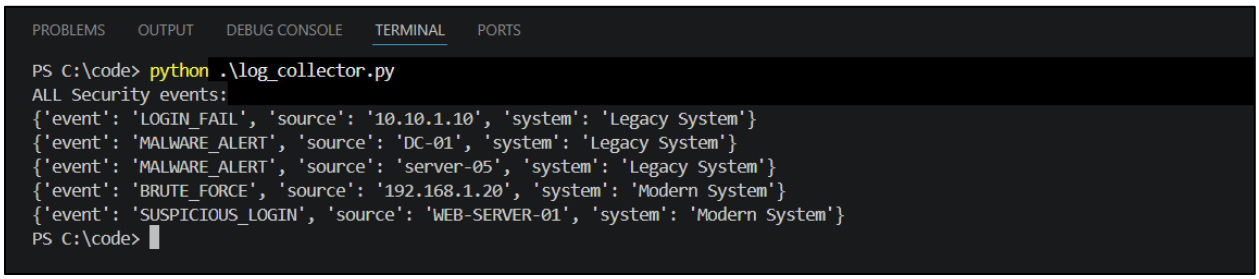
```
from abc import ABC, abstractmethod

# Target Interface
class SecurityEventSource(ABC):

    @abstractmethod
    def get_events(self):
        pass
#  Legacy System

class LegacyLogCollector:

    def read_old_logs(self):
        return [
            "LOGIN_FAIL|10.10.1.10",
            "MALWARE_ALERT|DC-01",
            "MALWARE_ALERT|server-05"
        ]
# LogAdapter:

class LogAdapter(SecurityEventSource):

    def __init__(self, legacy_collector):
        # store the legacy system in the adapter
        self.legacy_collector = legacy_collector

    def get_events(self):
```

```
        events = []
        legacy_logs = self.legacy_collector.read_old_logs()

        for log in legacy_logs:
            event_type, source = log.split("|")


            # convert it into the new format
            events.append({
                "event": event_type,
                "source": source,
                "system": "Legacy System"
            })
        return events

#Modern System
class ModernLogCollector(SecurityEventSource):
    def get_events(self):
        return [
            {
                "event": "BRUTE_FORCE",
                "source": "192.168.1.20",
                "system": "Modern System"
            },
            {
                "event": "SUSPICIOUS_LOGIN",
                "source": "WEB-SERVER-01",
                "system": "Modern System"
            }
        ]
#  create the log sources
legacy_source = LogAdapter(LegacyLogCollector())
modern_source = ModernLogCollector()
sources = [
    legacy_source,
    modern_source
]

# merge logs from the legacy and modern systems
all_events = []
for source in sources:
    all_events.extend(source.get_events())

# display the combined security events
```

```
print("ALL Security events:")
for event in all_events:
    print(event)
```

The result:

The legacy logs were successfully adapted and processed with the modern security events.



Bridge Pattern

In the second scenario, I applied the Bridge Pattern to a cyber security alert system. The alert types are separated from the delivery channels using the AlertChannel interface.

SecurityAlert uses the selected channel to deliver the message, while CriticalAlert adds a critical priority label. This allows alerts to use different delivery channels without creating a separate class for each combination.

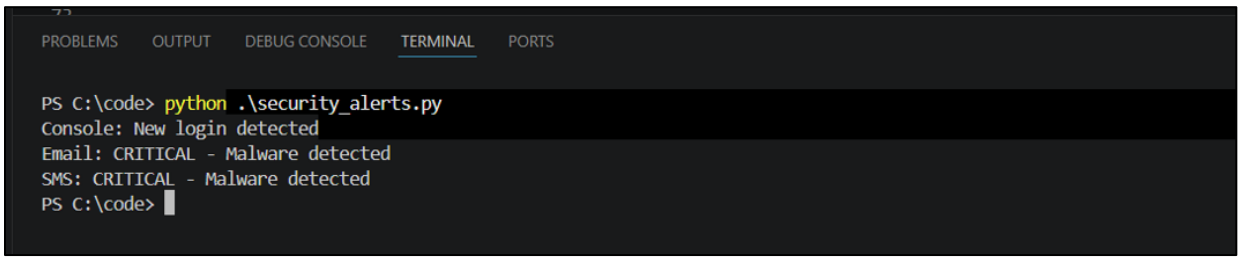
```
from abc import ABC, abstractmethod
class AlertChannel(ABC):

    @abstractmethod
    def send(self, message):
        pass
# Sends the alert to the console
class ConsoleChannel(AlertChannel):

    def send(self, message):
```

```
        print(f"Console: {message}")

# Sends the alert through SMS
class SMSChannel(AlertChannel):

    def send(self, message):
        print(f"SMS: {message}")

# Sends the alert through email

class EmailChannel(AlertChannel):

    def send(self, message):
        print(f"Email: {message}")

# -- Bridge:
# The alert does not know how the message is delivered It only uses the
AlertChannel interface.

class SecurityAlert:

    def __init__(self, channel):
        # Store the selected delivery channel
        self.channel = channel

    def notify(self, message):
        # Delegate the delivery to the channel
        self.channel.send(message)


# Critical Alert extends Security Alert and changes the alert behaviour without
changing the delivery channel.
class CriticalAlert(SecurityAlert):

    def notify(self, message):
        self.channel.send(f"CRITICAL - {message}")


#-- The alert type and the delivery channel can be combined independently.
# Normal security alert sent to the console
SecurityAlert(ConsoleChannel()).notify(
    "New login detected"
)

# Critical security alert sent through email
```

```
CriticalAlert(EmailChannel()).notify(
    "Malware detected"
)

# Critical security alert sent through SMS
CriticalAlert(SMSChannel()).notify(
    "Malware detected"
)
```

The result:

Security alerts were delivered through different channels using the same alert structure.



Composite Pattern

In the third scenario, I applied the Composite Pattern to a network asset inventory. The AssetComponent interface allows both individual devices and groups of devices to use the same show() method.

Device represents a single asset, while AssetGroup can contain devices or other groups. This allows network assets to be organised and displayed as a hierarchical structure through the same interface.

```
from abc import ABC, abstractmethod


#interface
class AssetComponent(ABC):
```

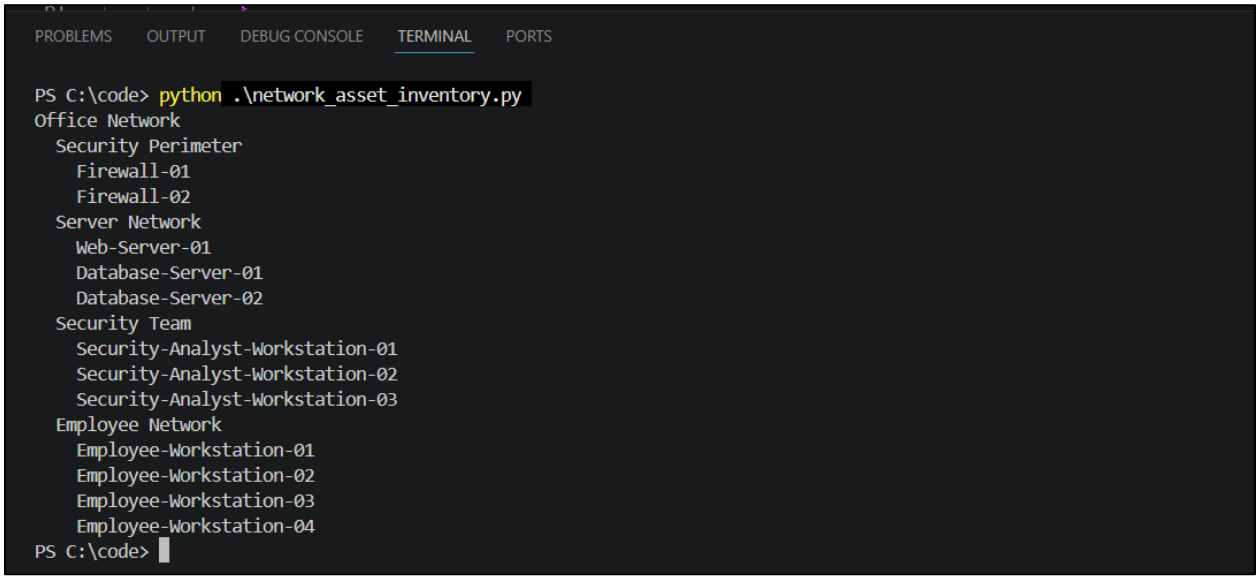
```
    @abstractmethod
    def show(self, indent=0):
        pass

# network device
class Device(AssetComponent):

    def __init__(self, name):
        self.name = name

    def show(self, indent=0):
        print(" " * indent + self.name)


# Composite  Devices or  AssetGroups
class AssetGroup(AssetComponent):

    def __init__(self, name):
        self.name = name
        self.children = []

    def add(self, component):
        self.children.append(component)

    def show(self, indent=0):
        print(" " * indent + self.name)

        for child in self.children:
            child.show(indent + 2)

# create network asset

firewall_01 = Device("Firewall-01")
firewall_02 = Device("Firewall-02")

web_server = Device("Web-Server-01")

database_01 = Device("Database-Server-01")
database_02 = Device("Database-Server-02")

security_workstation_01 = Device("Security-Analyst-Workstation-01")
security_workstation_02 = Device("Security-Analyst-Workstation-02")
security_workstation_03 = Device("Security-Analyst-Workstation-03")

employee_workstation_01 = Device("Employee-Workstation-01")
```

```
employee_workstation_02 = Device("Employee-Workstation-02")
employee_workstation_03 = Device("Employee-Workstation-03")
employee_workstation_04 = Device("Employee-Workstation-04")


#-- Create asset groups

# Firewall group
security_perimeter = AssetGroup("Security Perimeter")
security_perimeter.add(firewall_01)
security_perimeter.add(firewall_02)


# Server group
server_network = AssetGroup("Server Network")
server_network.add(web_server)
server_network.add(database_01)
server_network.add(database_02)


# Security team workstation group
security_team = AssetGroup("Security Team")
security_team.add(security_workstation_01)
security_team.add(security_workstation_02)
security_team.add(security_workstation_03)


# Normal employee workstation group
employee_network = AssetGroup("Employee Network")
employee_network.add(employee_workstation_01)
employee_network.add(employee_workstation_02)
employee_network.add(employee_workstation_03)
employee_network.add(employee_workstation_04)


# Main Composite
# Office Network contains several different asset groups

office_network = AssetGroup("Office Network")

office_network.add(security_perimeter)
office_network.add(server_network)
office_network.add(security_team)
office_network.add(employee_network)

# Display the complete network Asset
```

```
office_network.show()
```

The result:

Individual devices and asset groups were displayed as one hierarchical network structure.



Reference:

Gamma, E., Helm, R., Johnson, R. and Vlissides, J. (1994) *Design Patterns: Elements of Reusable Object-Oriented Software*. Reading, MA: Addison-Wesley.

Sarcar, V. (2022) *Java Design Patterns: A Hands-On Experience with Real-World Examples*. 3rd edn. New York: Apress.